CSA5112 - Cryptography and Network Security

17. DIGITAL SIGNATURE

AIM:

To implement a **Digital Signature using RSA and SHA-256** in Python and verify the authenticity and integrity of a message.

ALGORITHM:

1. Start the program.
2. Enter the message.
3. Generate an RSA public-private key pair.
4. Create a hash of the message using SHA-256.
5. Sign the message using the **private key**.
6. Verify the signature using the **public key**.
7. If verification succeeds, display **Signature Valid**.
8. Otherwise, display **Signature Invalid**.
9. Stop.

PYTHON PROGRAM:

First install the required library:

pip install cryptography

Then run:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generate RSA private key
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Get public key
public_key = private_key.public_key()

# Read message
message = input("Enter the message: ").encode()

# Create digital signature
signature = private_key.sign(
    message,
    padding.PKCS1v15(),
```

```
    hashes.SHA256()
)

print("\nDigital Signature Generated Successfully!")
print("Signature:", signature.hex())

# Verify signature
try:
    public_key.verify(
        signature,
        message,
        padding.PKCS1v15(),
        hashes.SHA256()
    )
    print("\nDigital Signature: VALID")

except Exception:
    print("\nDigital Signature: INVALID")
```

OUTPUT: